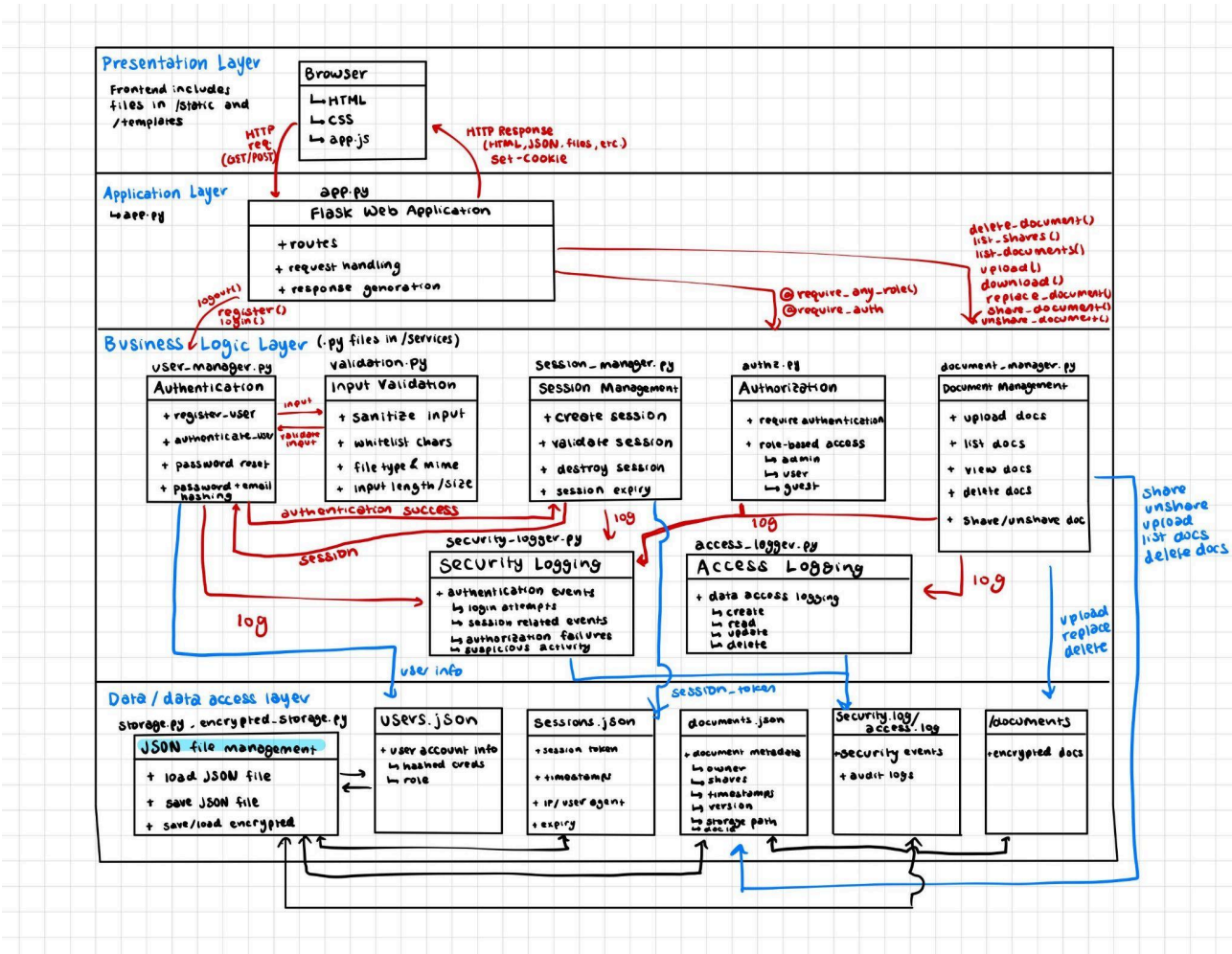
Architecture Overview

System Architecture Diagram

The system architecture consists of a presentation layer (front-end/client interface), an application layer (which contains the Flask application), a business logic layer (which contains all the services involved in the backend portion of the application), and a data/data-access layer (which includes the json files used for storage and python files used for loading/saving those files).

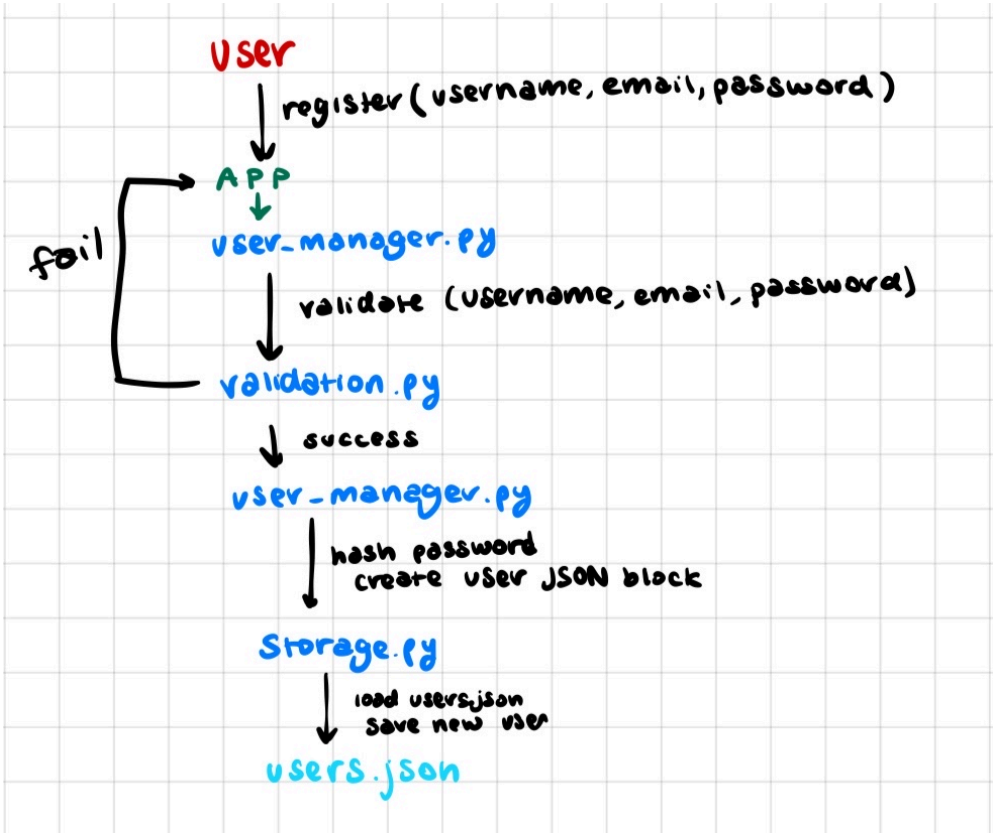


References:

- https://medium.com/@sagar.hudge/layers-in-software-architecture-c8cc16329ff6
- https://aws.amazon.com/what-is/architecture-diagramming/

Data Flow Diagrams

The diagram of the registration data flow is included below, and the rest of the data flow processes are simplified and described in text for legibility.

- **REGISTER**
- 

- **LOGIN**
  - user submits username/password → request goes to APP
  - APP handles requests, sends to user_manager.py
  - user_manager.py authenticate_user() function sends input to input_validation.py
  - clean input is sent back to user_manager() (or if ERROR, send back to APP and display error message)
  - if user_manager.py successfully authenticates the user, send info to session_manager.py
  - session_manager.py creates a session token and sends it to the APP to be put into a cookie for the user. The token hash is stored in sessions.json along with other information such as user, timestamps, IP address, and user agent

    - success message sent to the APP to be displayed to the user
- **DOCUMENT ACCESS**
    - user requests document by doc_id (accessible on document page for user), request sent to APP
    - APP first checks authentication & authorization for protected functions → sends to session_manager.py and authz.py to validate session and check for required roles
    - for UPLOAD
        - APP sends file info to validation.py to check file extension + magic bit + filename → send back to document_manager.py if safe.
        - send doc to encrypted_storage.py to be encrypted with Fernet cipher and stored in data/docs, store metadata in documents.json with storage.py
    - for READ
        - document_manager.py checks documents.json for document permission → send download request to APP

Other simplified data flows:

- sharing: owner of doc selects target+role → ACL / share list updated in documents.json
- admin view: admin requests logs/docs → RBAC check → permitted access returned

References:

- https://www.lucidchart.com/pages/data-flow-diagram

Component Descriptions

presentation layer:

- combination of HTML in /templates, and CSS/JS in /static

application layer:

- app.py: Flask app defines routes and connects frontend requests (from users) to backend logic (services listed below)

data logic/access layer (/services):

- user_manager.py: registration, authentication, password reset, user lookup, user management
- session_manager.py: creates, validates, and destroys sessions

- document_manager.py: upload, download, replace, delete, share, and document access checks
- validation.py: validates usernames, email addresses, passwords, and filenames, file extensions, etc
- security_logger.py: records security related events (authentication/authorization failures, errors, suspicious behavior, etc.)
- access_logger.py: records create/read/update/delete operations for document security/version control
- authz.py: authorization decorators enforcing RBAC and access to protected route/controller functions
- storage.py: load/save objects in JSON file
- encrypted_storage.py: encrypts/decrypts document contents before storage in JSON files

data layer

- users.json: contains user information
- documents.json: contains metadata for uploaded documents, including doc id, location, sharing info, version info, etc
- resets.json: contains hashed password reset tokens, and info about expiration/usage of resets
- sessions.json: contains hashed session tokens containing user, IP, user agent, etc.

Technology Stack Justification

For this project, I chose to use Python with Flask for the backend application development. I am a beginner in web application development and am already familiar with Python, so I thought it would be a good starting point. I used ChatGPT as a learning tool to help with starting the Flask application implementation, along with the starter codes provided in the project description. I found that it was a little similar to the JavaFX and Android Studio projects I've done before. I used bcrypt for password salting and hashing, and the cryptography library for encrypting the documents. I used Werkzeug for secure file and pathing.

As for the frontend development, I used HTML5 for the site structure, CSS3 for basic UI design, and a JavaScript file to provide the scripting of the site. I've done very basic work with

HTML, CSS, and JavaScript in the past, and given the time constraints, I did not think I should try to learn how to use Bootstrap and CSS for styling.

I opted to use JSON files to keep track of user accounts, active sessions, password reset tokens, and document IDs. The documents themselves are kept as encrypted files in the /data/docs folder. The logs are kept in the /data/logs folder. All create/read/update/delete operations are stored in access.log (so any events related to document/data access), and all security-related auditing is stored in security.log (including suspicious behavior, authorization/authentication fails, etc)

Threat Model

Asset Identification

Assets include information such as the user's credentials (password hash, username, email address), uploaded docs and their contents, session tokens and active session records, password reset tokens, document access control lists, access logs, and security logs.

Threat Enumeration

Threat actors include external attackers attempting unauthorized access, malicious authenticated users trying to access other users' documents, careless insiders misusing legitimate access, automated bots submitting malicious input, or credential stuffing.

Vulnerability Assessment

Potential vulnerabilities could include (but are not limited to) the following:

a. Weak passwords during registration (or just generally invalid input)
b. Unauthorized access to documents/user info without proper access checks
c. Injection attacks through form inputs (XSS / command injections)
d. Path traversal through malicious filenames
e. Malicious files (binary files, mismatched magic bit and file extension, MIME types, etc.)
f. Session hijacking or reuse of stale sessions
g. Exposure of sensitive data if files are stored unencrypted
h. Insufficient logging, making attacks/suspicious behavior harder to detect

i. Abuse of admin functionality if role checks fail

The application accounts for all of the above vulnerabilities and attempts to remediate/prevent attacks on all assets.

Attack Scenarios

- XSS attempt: attacker submits JavaScript in the input field (maybe username field, hoping it will execute in the browser)
- Path traversal attempt: attacker tries to change the URL to get to the desired page
    - upload / request filename such as ../../users.json to escape doc storage directory (to get to data)
- Privilege escalation attempts: guest attempts to go to the admin dashboard or attempts to upload a file
- unauthorized doc access: logged-in user manipulates a doc ID to access a file that isn't shared with them
- session abuse: attacker attempts to reuse an invalid/expired session token to gain access

Additional potential attacks are listed in the penetration testing document.

threat	security control
brute force attacks	account lockout, rate limiting by IP, strong password
cred stuffing	rate limiting, strong password
unauthorized doc access	RBAC decorators, doc-level ACL checks (ACL is attached to doc metadata)
privilege escalation	decorators used to enforce roles (require_auth, require_any_role)
XSS attacks	input sanitization/validation, HTML escaping, CSP (content security policy header)
command injection	input validation (and no shell execution is used for this)
path traversal	secure filename and path validation
malicious file upload	extension + MIME + magic byte check. very basic pattern check
session hijacking	secure cookies, session timeouts, and user-agent binding

data exposure (at rest)	encryption using Fernet

Risk Prioritization

Risks are prioritized based on impact to CIA triad.

- High: any kind of data exposure including unauthorized access, authentication bypass, privilege escalation
- Medium: session abuse (invalid/expired session tokens), path traversal, XSS/command injection
- Low: password reset abuse, registration spamming, botting

Security Controls

The app uses defense-in-depth by applying multiple layers of security controls (like authentication, authorization, encryption, sessions, logging, etc.) to reduce the likelihood/impact of potential attacks.

Requirement 1: User Authentication

Control Description

User authentication ensures that the application verifies that the user is who they say they are and makes sure that they are authorized to perform whatever action they are trying to take. First, they must register with a valid username, email, and a strong password. The username and email must be unique. When logging in, the user is authenticated via username and password. If the user forgets their password, they may reset their password through a reset link sent to their email address (which also would be a form of authentication- they must have access to the email in order to reset their password).

Implementation Approach

Authentication is implemented using a simple username-and-password system. If they forget their password, a reset link is generated with a unique reset token sent to the user's email (which is also a form of authentication, as only the user should have access to the email). Every

function in the application that requires authenticated users checks for authentication with a @require_auth decorator function (ensures users are logged in with a valid session).

Testing Methodology

- register with a valid username, email, and password
- attempt weak password
- attempt duplicate username/email
    - keep in mind that email ignores case
- log in with the correct password
- attempt to log in with incorrect password
    - repeat 5 times for lockout
    - verify login works after 15 minutes (attempts reset)
- reset password & verify old password no longer works
    - password reset cannot be spammed, but generic success message is printed every time user clicks button. Audit log will let you know email was suppressed (even though generic success message is printed).
    - password reset email delivery uses SMTP if environment variables are configured. If SMTP is not configured, the application prints the reset link to the terminal for testing purposes.
- verify new password works after reset

Known Limitations

For this project, due to time constraints, authentication is implemented only through username and password. Perhaps in the future, I could implement MFA using an email code, but for now, it will be submitted as is. Password reset delivery email and token handling are a little simplified. While I did hash the password reset token (in case someone gets access to the password_resets.json file), it's just a simple link (no further verification needed other than access to the link).

Mitigation Strategies

- strong password requirements
- input validation

- bcrypt hashing for passwords / SHA256 hashing for tokens (session and reset)
- reset token validation
- security logging of authentication events (for monitoring)

Requirement 2: Access Control

Control Description

The app uses a role-based access control system, with three roles: guest, user, and admin. Guests are the default role upon registration. They may only view shared content. An admin can manage accounts- they can promote/demote guests and users. If a guest is promoted to a user, they may create content, edit/delete their own content, and access content that is shared with them. Admins can do everything a user can do, but again, they can manage users (promote/demote), and they can view ALL documents, even if it was not shared with them. They may also view security and access logs, but they cannot edit or delete the logs.

The documents themselves are managed with a discretionary access control system in the form of access control lists attached to the document information (share lists are attached to each document in the documents.json file). The owner of the document is the user who uploaded the original file. They can edit the document, share/unshare with others as either editor or viewer, delete the document, and view the access list for the document. Editors can replace the document with another version or simply view the content (by downloading it). Viewers can only view the content (by downloading it).

Implementation

I implemented the Role-Based Access Control (RBAC) system with decorator functions. Any functions that require a certain role use the @require_role function. The roles (guest, user, admin) were implemented as described above.

The discretionary access control (DAC) system was implemented using a document_manager file that can access the documents.json file to make changes to content, shares, etc., for a given document. Any functions that require a certain access level (for the document) were checked with functions such as can_view, can_edit, can_share, etc. The access roles (owner, editor, viewer) were implemented as described above.

Testing Methodology

- confirm guest cannot create documents and cannot be assigned the “editor” role on documents
- confirm admin/user can upload and manage their own documents
- confirm viewer can read but not edit
- confirm editor can replace, but not delete/share with others
- confirm non-shared users cannot access the document
- confirm admin can view all documents and log files
- confirm admin cannot **edit** documents that are not shared with them (only view) and **cannot edit** log files
- confirm admin can promote guest to user and demote user to guest
- confirm that if a user is demoted to a guest, they are demoted to viewers for any documents they have editing access to
    - for documents owned, they can no longer replace the document/share it, but it will remain up

Known Limitations

ACLs are stored in documents.json rather than a capabilities list. As a result, it’s hard to see all the access that someone has (by user) and manage it accordingly. Additionally, the role hierarchy is very simple. It only includes guest, user, and admin. It should have another role for a “database manager” who can see audit logs, but due to time constraints, only admins can view the logs, which gives a little too much power to the admins.

Mitigation Strategies

- role decorators for centralized authorization checks
- explicit perm checks for every doc action
- error messages are vague for users to avoid revealing any sensitive info (such as existing docs/user info), though the audit logs contain the accurate information.
    - instead of saying “unauthorized” on attempt to access unauthorized materials, the error message just says error: not found.

Requirement 3: Input Validation & Injection Prevention

Control Description

User input is never to be trusted. Any form of input is restricted through drop-downs or simple buttons. Any text boxes are put through input validation and character escaping. Allowed inputs are whitelisted, length limits are put into place, and all input is type checked. The app includes XSS prevention (Flask automatically escapes HTML characters, which prevents injected scripts that involve HTML characters. Additionally, all input is sanitized/validated with regex based on requirements for each input box), command injection prevention, and path traversal prevention. When rendering things like file names back into the interface, templating and sanitization prevent embedded scripts from executing (file names would not be allowed to be scripts due to limited file extensions and characters). Tested inputs like <script>alert('XSS')</script> were treated as text rather than executable code. Path traversal attempts, such as ../../passwds, do not allow access outside the allowed storage path (site displays error and says not authorized). Made sure pathing was safe using os.path and joining.

Implementation

- Did not use Jinja templates but made sure to escape using implemented functions in validation.py
    - Flask also automatically escapes HTML characters
    - HTML5 also checks for some invalid input (like for emails)
- username validation using whitelist rules and length limits → only validate allowed characters
- email validation using regex/full-match to check the format of the email
- password strength validation
- filename sanitization using secure filename handling
- type and length checks wherever it's relevant
- html escaping and securely rendering templates to reduce XSS risk
- rejecting invalid/suspicious inputs → prevent command injection

Testing Methodology

Check for the following and handle accordingly:

- XSS payload in text fields
- XSS payload in filenames
- malformed email
- input length checks
- invalid filename (script, invalid characters, length, etc)
- path traversal payloads like ../../secret.txt
- file extension mismatch
- suspicious file contents

Known Limitations

Regex-based validation can miss edge cases if I didn't account for certain attacks/payloads. Also, if I misconfigured the regex pattern, valid inputs could be rejected (for example, my email formatting may be too strict, but I went off of guidelines I found online for valid email formatting). Complex HTML sanitization isn't implemented beyond safe rendering/escaping, but it looks like HTML5 and Flask automatically scan for bad inputs. The malware check is very simple- it only includes a basic magic bit check and makes sure the file extension and actual type match, and checks for patterns like <script> or <?php>.

Mitigation Strategy

- server-side validation (validation.py) on user inputs through regex and built-in Flask rendering
- safe template rendering through Flask
- secure filename handling (werkzeug)
- default-deny behavior for input (if it doesn't match the regex pattern, it will be rejected)
- add malware scanning software (better to find a software that focuses on this rather than trying to implement every scenario/attack)

Requirement 4: Encryption

Control Description

All sensitive data at rest is encrypted with the Fernet cipher and key, or with bcrypt, SHA256, and the cryptography library. Passwords are salted and hashed, documents are encrypted, user emails and reset/session tokens are stored as hashes, etc. Data in transit is implemented in the application code through TLS by enforcing HTTPS redirection. However, due to local deployment through Flask with a self-signed certificate, the redirection isn't working (cannot view the app with HTTP connection, but if manually changing to HTTPS connection, it will work). In a production environment, we would want the app to work with HTTP and HTTPS URLs so that HTTP redirects to HTTPS. Sensitive stored data is protected using password hashing

Implementation Approach

- Passwords are salted and hashed using bcrypt to prevent rainbow table attacks.
- Uploaded documents are encrypted before being written to disk using Fernet symmetric encryption from the cryptography library.
    - Decryption only occurs when an authorized request is made to access the document (only those who have viewing permissions may view the document)
- Fernet key-file is generated LOCALLY if it doesn't exist (through the config.py file). If the key file is changed or removed, any stored documents or existing sessions cannot be decrypted/validated.

Testing Methodology

- verify documents saved on disk are unreadable without decryption
    - also verify that if any keys change, everything is no longer readable
- verify authorized download correctly decrypts
- verify unauthorized users cannot trigger document retrieval
    - (even if they use the URL with the right doc_id)
- verify password storage contains bcrypt hashes, not plaintext
- verify reset and session tokens are hashed in storage

Known Limitations

- Key management is simplified for this project. It is stored locally-- could be bad if anyone gets access to the root directory, though all path traversal attempts are restricted.
- JSON file storage is fine for small-scale project, but a database would be better for different applications → local file-based storage != database security
- transport encryption depends on the deployment environment. While I don't directly handle TLS, I made sure to redirect http connections to https connections.
    - need to run with HTTPS, which is implemented in app.py, but that requires deployment on a server rather than local deployment through Flask

Mitigation Strategies

- encryption at rest for documents and sensitive info (like the JSON files)
- hashed passwords and tokens
- restricted access paths
- it's recommended that secret.key is stored securely instead of just kept locally…

Requirement 5: Session Management

Control Description

Sessions are created on successful authentication (login). Every function calls for session validation. When a session times out, it is destroyed. Logout also triggers session destruction

Implementation Approach

After successful login, the system creates a session token associated with the browser, user, IP, etc. All protected routes/functions validate the session before allowing access. Sessions are destroyed on logout, and session timeouts are enforced to reduce the risk of stale sessions being reused. Session tokens are stored as hashed tokens to prevent session hijacking.

Testing Methodology

- log in and access protected routes
- log out and confirm protected routes fail afterward
- use an invalid session token

- use an expired session
- attempt access without authentication
- attempt using token stored in JSON file (will fail because it is hashed)

Known Limitations

- Session storage in JSON is fine for a small-scale project like this, but could cause problems (not sure if ACID is really implemented here, if lots of people are creating new sessions by logging in → what happens if two people try to register with the same username at the same time??)
- sessions are not bound to IP (in case they are using a mobile browser, which might swap between wifi/cellular data. They are bound to the user-agent, though.

Mitigation Strategies

- Session validation on protected endpoints
- Explicit session destruction on logout
- Timeout enforcement
- Security logging for session creation/destruction
- Hashed session tokens

Requirement 6: Security Headers

Control Description

The application uses the HTTP response headers included in the project description and starter code. The headers include Content-Security-Policy; X-Frame-Options: DENY; X-Content-Type-Options: nosniff; X-XSS-Protection: 1; mode = block; Referrer-Policy: strict-origin-when-cross-origin; Permissions-Policy: disabled geolocation, microphone, camera; Strict-Transport-Security: max-age, includeSubDomains

Implementation

The implementation was enforced using the code provided to us in the project description/write-up.

Testing Methodology

- inspect response headers using browser inspect tool (since security header site would require deploying the app)
    - confirm header presence on major pages (login, register, etc)
- attempt iframe embedding if clickjacking prevention is enabled
- verify that the uploads cannot be fooled by incorrect MIME extensions
- use inspect tool console to insert code for redirects or iframes (included in penetration testing document)

Known Limitations

- This app only includes the response headers required by the project guidelines. Any other headers have not been implemented
- HSTS only works meaningfully under HTTPS deployment. While the header is present in app.py, and I added the code for HTTP to HTTPS redirect, the site won't actually reflect the changes. This is due to the Flask app local deployment settings. The page is not deployed through HTTPS

Mitigation Strategies

- restrictive header defaults
- consistent application-wide header injection
- minimal external content dependencies
- recommended to deploy the app on a dedicated server, not Flask local deployment

Requirement 7: Logging & Monitoring

Control Description

Security and access events are logged in JSON files. These include failed authentication attempts, authorization failures, password changes/reset requests, account lockouts, data access (create, read, update, delete), session creation/destruction, security configuration changes, input validation failures, suspicious activity (rapid requests, invalid tokens), etc

Implementation

The application uses separate logs for access events and security events as described below to allow admins to monitor suspicious activity in the security logs and monitor general use in the access logs. Logs include timestamps, error details, severity, and resources being accessed.

- access logging
    - info: success of create, read, update, delete, share/unshare docs (resource, action)
- security logging
    - info: login success, account creations, session creations, etc
    - warning: login failed (username, reason), access denied (resource, insufficient privileges)
    - error: account locked
    - critical: sus activity (rapid requests, invalid tokens)

Testing Methodology

- Failed login attempts
- Successful login/logout
- Unauthorized document access attempt
- Invalid token submission
- Password reset
- Admin actions (viewing audit logs, viewing all docs, managing users)
- create, read, update, delete, share, operations on documents in access.log
- Password reset cannot be spammed, but same generic success message is printed. The log will let you know email was suppressed

Known Limitations

- Log review is manual (so no alert system)
- No separate role for log access (viewing permissions were given to admin)
- Logs stored in JSON files rather than any log manager systems / SIEM
- Logs are not deleted or managed. They are going to just keep building up

Mitigation Strategies

- Separate audit categories
- Security event capture
- Log access is limited- don't allow log modification/tampering

Data Protection

Data Classification

- Highly sensitive (PII, credentials, authentication): password hashes, emails, reset tokens, session tokens, encryption keys
- Sensitive: uploaded documents, usernames, access control lists
- Internal: logs, document metadata
- Public/low sensitivity: static frontend files

Encryption Methods

- bcrypt for passwords
- SHA256 hashing for emails and tokens (session/reset)
- Fernet (with local key file) for encrypted document contents and email encryption
- TLS depends on HTTPS, which I attempt to enforce in app.py, though it will only be meaningful in actual deployment

Key Management

- Document encryption depends on a symmetric key used by the application. For this class project, key management is simplified. It relies on the secret.key file generated by config.py, and the session signing "secret" generated and stored in the environment secrets.
- On a larger scale (outside the project), encryption keys should be stored separately from application data, rotated periodically, and protected using a key management system/service.

Secure Deletion Procedures

When a document is deleted, both its metadata entry (documents.json) and the stored encrypted file are removed from the application's data directories. Since this project uses local file storage, deletion just entails deleting the encrypted document (won't be decrypted even if someone finds the encrypted document since the key stays private) and the document data (owner, shares, etc.) → the JSON file is overwritten by a version that doesn't have the deleted document metadata.